CS 305 Project One Template

Document Revision History

Version	Date	Author	Comments
1.0	3/20/2026	Duane Day Jr	

Client

Instructions

Submit this completed vulnerability assessment report. Replace the bracketed text with the relevant information. In this report, identify your security vulnerability findings and recommend the next steps to remedy the issues you have found.

- Respond to the five steps outlined below and include your findings.
- Respond using your own words. You may also include images or supporting materials. If you include them, make certain to insert them in the relevant locations in the document.
- Refer to the Project One Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Duane Day Jr

1. Interpreting Client Needs
Determine your client's needs and potential threats and attacks associated with the company's application and software security requirements. Consider the following questions regarding how companies protect against external threats based on the scenario information:

- What is the value of secure communications to the company?
- Are there any international transactions that the company produces?
- Are there governmental restrictions on secure communications to consider?
- What external threats might be present now and in the immediate future?
- What modernization requirements must be considered, such as the role of open-source libraries and evolving web application technologies?

Artemis Financial is a company that works with sensitive financial data such as savings, investments, and retirement accounts. Because of this, secure communication is extremely important. If the system is not secure, customer data could be stolen or changed, which would damage trust and possibly cause legal problems.
Secure communication protects data while it is being transferred between users and the system. This includes encryption and secure APIs. Without protection, attackers could intercept information like account details or personal data.
Artemis Financial likely supports international transactions since financial companies often serve users in different regions. This means they must follow different security regulations depending on the country, such as data protection laws.
There may also be government requirements related to encryption and privacy. These rules make sure companies properly protect customer information.
External threats include hackers, phishing attacks, and vulnerabilities in the web application. In the future, threats could become more advanced with automated attacks and newer malware.
Modernization is also important. The company must keep its system updated and make sure all libraries and frameworks are secure. Outdated software can introduce vulnerabilities into the system.

2. Areas of Security
Refer to the vulnerability assessment process flow diagram. Identify which areas of security apply to Artemis Financial's software application. Justify your reasoning for why each area is relevant to the software application.

- Input Validation (Secure Input and Representations)
  All user input must be validated to prevent malicious data or injection attacks.
- APIs (Secure API Interactions)
  The REST API must be secured to prevent unauthorized access to financial data.
- Cryptography (Encryption Use and Vulnerabilities)
  Sensitive data must be encrypted to protect it from being exposed.
- Client/Server (Secure Distributed Computing)
  Communication between client and server must be secured using HTTPS.

- Code Error (Secure Error Handling)
  The application should not expose detailed error messages.
- Code Quality (Secure Coding Practices/Patterns)
  Clean and secure coding practices reduce vulnerabilities.
- Encapsulation (Secure Data Structures)
  Sensitive data should be protected within classes.

3. Manual Review

Continue working through the vulnerability assessment process flow diagram. Identify all vulnerabilities in the code base by manually inspecting the code.

- The project uses Spring Boot 2.2.4.RELEASE, which is outdated and may contain security risks.
- The dependency bcprov-jdk15on:1.46 is very old and has known vulnerabilities.
- In GreetingController.java and CRUDController.java, user input is accepted without validation, which could allow malicious input.
- There is no authentication or authorization, meaning anyone can access the application endpoints.
- In DocData.java, the database uses hardcoded credentials (root / root), which is insecure.
- Error handling uses printStackTrace(), which may expose system details.
- The application does not enforce HTTPS for secure communication.
- In customer.java, sensitive data like account balance is not fully protected with proper encapsulation.

4. Static Testing

Run a dependency check on Artemis Financial's software application to identify all security vulnerabilities in the code. Record the output from the dependency-check report. Include the following items:

- The names or vulnerability codes of the known vulnerabilities
- A brief description and recommended solutions provided by the dependency-check report
- Any attribution that documents how this vulnerability has been identified or documented previously

  The library bcprov-jdk15on:1.46 is outdated and has known security vulnerabilities. The use of Spring Boot 2.2.4. RELEASE also introduces risk because it may rely on older vulnerable components.

Example vulnerability:
Older versions of dependencies such as BouncyCastle and Spring components are known to have CVE-related vulnerabilities that can expose systems to attacks.
Recommended fixes:

- Update all dependencies to the latest versions
- Upgrade Spring Boot to a newer supported version

- Regularly run dependency check scans

Attribution:
OWASP Dependency Check tool (uses the National Vulnerability Database)

5. Mitigation Plan

Interpret the results from the manual review and static testing report. Then identify the steps to mitigate the identified security vulnerabilities for Artemis Financial's software application.

To reduce the vulnerabilities in the Artemis Financial application, several improvements should be made.
First, outdated dependencies should be updated. The Bouncy Castle library should be upgraded from version 1.46 to a newer secure version. Spring Boot should also be updated to a current supported version. After updating, the dependency check tool should be run again to confirm that vulnerabilities are removed.
Next, input validation should be added to all controller methods to prevent malicious input. Hardcoded database credentials should be removed and replaced with secure configuration methods such as environment variables.
Error handling should be improved so that sensitive system details are not exposed. Instead of printing stack traces, the application should log errors securely.
Authentication and authorization should be added so only authorized users can access the system. HTTPS should also be enforced to secure communication between client and server.
Finally, sensitive data fields should be properly encapsulated using private access and controlled methods. These improvements will help protect customer data and reduce the risk of attacks.